
Short-Term Urban Traffic Speed Prediction Using Machine Learning

Ouyi Yang

Department of Electrical and Computer Engineering
University of Toronto
ouyi.yang@mail.utoronto.ca

Qiwen Lin

Department of Electrical and Computer Engineering
University of Toronto
qw.lin@mail.utoronto.ca

Yuhao Cao

Department of Electrical and Computer Engineering
University of Toronto
davidyh.cao@mail.utoronto.ca

Abstract

This project presents a machine learning framework for short-term urban traffic speed prediction using real-world traffic and weather data from the University of Toronto area. We predict average speed at 15-minute intervals as a regression task. Baseline models (XGBoost, LightGBM, KNN, ResNet MLP) showed that models with similar capacity produce comparable results, so we focused on feature engineering—expanding 19 raw features to 45 engineered features—which reduced MAE by 16–23% across all models. After tuning, XGBoost was selected as the final model: it matched the ResNet MLP in accuracy (MAE 3.180 vs. 3.172 km/h) while training $3\times$ faster.

Attestation of Teamwork

Ouyi Yang: Developed and evaluated baseline models (XGBoost, LightGBM, KNN, Random Forest), ensured consistent feature usage across models to enable fair comparison and contributed to report writing.

Qiwen Lin: Developed and evaluated neural network models (MLP and ResNet), implemented GPU acceleration for training, and contributed to report writing.

Yuhao Cao: Prepared the dataset and implemented the feature engineering pipeline, including temporal, lag-based, rolling statistical, and weather-related features. Also contributed to report writing.

All members contributed to discussions and final model selection.

1 Introduction

Accurate short-term traffic speed prediction is important for reducing congestion and enabling proactive traffic management. This project predicts the average traffic speed at 15-minute intervals on urban

road segments near the University of Toronto campus using machine learning. We first compared several baseline models, then focused on feature engineering—which proved more impactful than model complexity—before selecting XGBoost as the final model.

2 Preliminaries and Problem Formulation

The task is formulated as a regression problem: given historical traffic and weather data at 15-minute intervals, predict the average speed at the next time step. Models are evaluated using:

- **MAE**: mean absolute prediction error (primary metric)
- **RMSE**: root mean squared error (penalizes large errors)
- **R^2** : proportion of variance explained

3 Design

The system follows a four-stage pipeline: data preprocessing, feature engineering, model training, and evaluation. The design was guided by two key observations made during early experiments.

3.1 Overall Architecture

1. **Preprocessing**: merge traffic sensor records (2.1M rows) with hourly weather data (54K rows) into a single time-aligned dataset of 1.58M rows.
2. **Feature engineering**: expand from 19 raw columns to 45 features by adding temporal encodings, lag values, rolling statistics, and weather indicators.
3. **Model training**: train on 80% of the data chronologically, validate on 10%, and test on 10%.
4. **Evaluation**: compare models using MAE, RMSE, and R^2 on the held-out test set.

3.2 Key Design Decisions

Why we focused on features over models. When we trained the baseline models on raw features, XGBoost, LightGBM, and ResNet MLP all ended up around MAE ≈ 3.85 . Since very different models gave the same result, we figured the problem was the features, not the models. Adding engineered features dropped MAE by 16–23% for every model, which confirmed this.

Avoiding data leakage. Because this is time-series data, we used a chronological 80/10/10 split instead of random splitting. We also shifted all lag and rolling features by at least one step so the model never sees future data during training.

Why we picked XGBoost. After tuning, XGBoost and ResNet MLP got almost the same accuracy (MAE 3.180 vs. 3.172), but XGBoost trained in 49 seconds compared to 139 seconds for ResNet MLP. Since the accuracy difference is negligible, we went with the faster model. The architecture is shown in Figure [1](#).

4 Methodology

4.1 Data Sources and Preprocessing

Traffic data were obtained from the Toronto Open Data Portal [5], containing about 2.1 million records across 1,947 road segments at 15-minute resolution from 2020 to 2025. Each record has counts across 14 speed bins, which we converted to a weighted average speed. Segments with fewer than 72 hours of data were removed to reduce noise.

Weather data (temperature, wind speed, precipitation, visibility) were obtained from Environment Canada [6], containing 54K hourly records from 2020 to 2026. We merged the weather data with traffic data by matching timestamps. The final merged dataset has 1.58 million rows and 45 features after engineering.

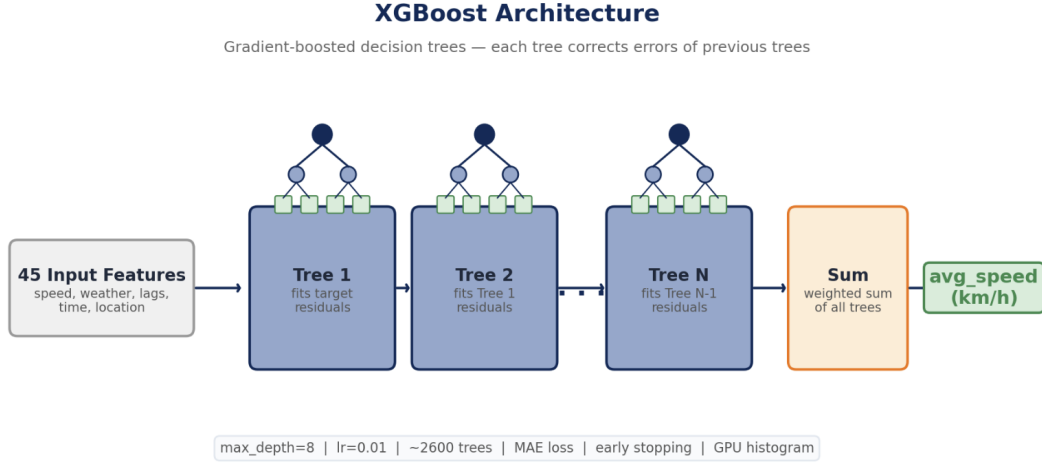


Figure 1: XGBoost architecture: each tree corrects the residual errors of previous trees. The final prediction is a weighted sum of approximately 2,600 trees.

4.2 Feature Engineering

We expanded the original 19 raw features to 45 engineered features. The added features can be grouped into four categories:

- **Temporal:** hour of day, day of week, month, sin/cos cyclical encodings, rush-hour and weekend flags.
- **Weather:** temperature, wind speed, precipitation, visibility, and binary flags for rain, snow, and fog.
- **Lag and rolling:** speed and volume deviation lags going back 1 to 16 time steps, plus rolling mean and standard deviation over multiple windows. All rolling windows are shifted by 1–2 steps to avoid look-ahead bias.
- **Historical average:** mean speed per segment/hour/day-of-week, computed from training data only and applied to validation and test sets through lookup to prevent leakage.

4.3 Models

We compared six models spanning three families:

- **Tree-based:** XGBoost [1] (gradient-boosted trees), LightGBM [2] (histogram-based leaf-wise boosting), and Random Forest (excluded due to impractical training time on 1.58M rows).
- **Distance-based:** KNN (k -nearest neighbors regression).
- **Neural networks:** MLP (fully connected) and ResNet MLP (with residual skip connections for better gradient flow), both GPU-accelerated.

5 Numerical Experiments

5.1 Experimental Setup

All experiments were run on a dataset of 1.58 million records from 2020 to 2025, covering 1,947 road segments near the University of Toronto St. George campus at 15-minute intervals. The dataset was split chronologically into 80% training, 10% validation, and 10% test to prevent data leakage. All models were evaluated on the same held-out test set using MAE, RMSE, and R^2 .

5.2 Baseline Comparison

We established two tiers of baselines: three naive methods (Global Mean, Historical Average, Persistence) and four ML models trained on 19 raw features. Persistence (MAE 4.193) serves as our main benchmark. Results are shown in Table 1.

Table 1: Baseline results on the test set. Naive baselines use no training; ML baselines use 19 raw features with no engineering.

Model	MAE (km/h)	RMSE	R ²	Train Time (s)
<i>Naive baselines</i>				
Global Mean	9.596	—	−0.341	—
Historical Average	8.403	—	−0.084	—
Persistence	4.193	—	0.612	—
<i>ML baselines (raw features)</i>				
XGBoost	3.840	5.566	0.705	2.5
LightGBM	3.954	5.732	0.687	119.2
KNN	5.042	6.808	0.559	5.1
ResNet MLP	3.858	—	0.698	47.9

All ML models except KNN beat Persistence even without feature engineering. XGBoost and ResNet MLP gave the best results (MAE $\approx$ 3.85).

5.3 Impact of Feature Engineering

Table 2 shows the MAE at each stage. Feature engineering (19→45 features) gave a 16–23% MAE reduction across all models—the most impactful step. Hyperparameter tuning added less than 0.5% further improvement.

Table 2: MAE (km/h) across model development stages.

Stage	XGBoost	LightGBM	KNN	ResNet MLP
Raw (no engineering)	3.840	3.954	5.042	3.858
+ Feature engineering	3.194	3.308	3.889	3.280
+ Hyperparameter tuning	3.180	—	—	3.172
Total improvement	17.2%	16.3%	22.9%	17.8%

5.4 Final Model Configuration and Results

We used early stopping based on validation MAE. Training stopped at around 2900 rounds out of 5000. Table 3 lists the hyperparameters and Figure 2 shows the learning curve. Both curves converge quickly and the narrow gap between train MAE (2.768) and validation MAE (2.723) confirms minimal overfitting.

Table 3: Final XGBoost configuration.

Parameter	Value
Estimators	5000 (stop ~ 2900)
Learning rate	0.01
Max depth	8
Subsample	0.8
Col sample	0.8
Reg α/λ	0.1 / 1.0
Objective	MAE
Tree method	hist (GPU)

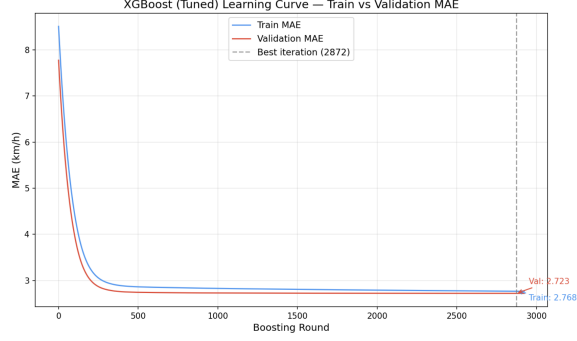


Figure 2: Train vs. validation MAE per boosting round. The narrow gap confirms minimal overfitting.

The final test set results are shown in Table 4.

Table 4: Test-set results: tuned models vs. Persistence baseline.

Model	MAE	RMSE	R^2	Train Time (s)
Persistence (baseline)	4.193	—	0.612	—
XGBoost (tuned)	3.180	4.775	0.781	49
ResNet MLP (tuned)	3.172	—	0.781	139

The final XGBoost model achieved MAE 3.180 km/h, RMSE 4.775, and R^2 0.781. This is a 24.4% improvement over the Persistence baseline (MAE 4.193).

5.5 Comparison with External Benchmark

Hammoumi et al. [7] report 96% accuracy on binary congestion classification using Random Forest on 9,847 samples. Converting our continuous predictions to binary labels yields 93.3% accuracy on 200K samples—comparable classification performance on a dataset $20\times$ larger, while also providing continuous speed estimates.

6 Discussion

Two main findings emerged. First, feature engineering was the most important factor: the 16–23% MAE reduction held across all model families, including KNN, confirming that the gains came from better input representation rather than model-specific tricks. The lag and rolling features capture temporal autocorrelation that raw features miss.

Second, a well-tuned XGBoost matched the ResNet MLP (MAE 3.180 vs. 3.172, same R^2) while training $3\times$ faster. This suggests that at this dataset size, the bottleneck is in data and features rather than model capacity.

7 Conclusions

XGBoost with 45 engineered features achieved MAE 3.180 km/h and R^2 0.781 on 1.58M real sensor records—a 24.4% improvement over the Persistence baseline while training in under a minute. The main lesson is that feature engineering matters more than model complexity for tabular traffic data. Future work could explore multi-step forecasting, spatial graph models like DCRNN [3], and online learning to handle evolving traffic patterns.

References

- [1] T. Chen and C. Guestrin (2016). XGBoost: A Scalable Tree Boosting System. *Proc. KDD 2016*, pp. 785–794.
- [2] G. Ke et al. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in NeurIPS* 30.
- [3] Y. Li et al. (2018). Diffusion Convolutional Recurrent Neural Network: Data-Driven Traffic Forecasting. *ICLR 2018*.
- [4] B. Yu, H. Yin, and Z. Zhu (2018). Spatio-Temporal Graph Convolutional Networks: A Deep Learning Framework for Traffic Forecasting. *IJCAI 2018*, pp. 3634–3640.
- [5] Toronto Open Data Portal. <https://open.toronto.ca/>
- [6] Environment Canada Historical Climate Data. <https://climate.weather.gc.ca/>
- [7] L. Hammoumi, S. Farah, M. Benayad, M. Maanan, and H. Rhinane, “Leveraging machine learning to predict traffic jams: Case study of Casablanca, Morocco,” *Journal of Urban Management*, vol. 14, no. 3, 2025. <https://doi.org/10.1016/j.jum.2025.02.004>

AI Usage

Artificial intelligence tools were used as supportive tools during this project.

AI was primarily used to assist in formatting the report into proper $\text{\LaTeX}$ structure and improving written clarity by revising informal expressions into more formal academic language. All technical content, experimental results, and interpretations were verified and written by the team.

In addition, AI-assisted workflows were used to automate the execution of multiple model configurations during hyperparameter tuning. This helped improve efficiency when testing different parameter combinations across models. However, all model selection decisions, feature engineering strategies, and performance evaluations were designed and validated manually by the team.

No AI tools were used to generate experimental results, fabricate data, or replace understanding of the implemented algorithms. All models were implemented, trained, and evaluated by the team using standard machine learning libraries.

Contest for Information Sharing

As part of this course, we may share selected project materials (e.g., reports, presentation slides, and presentation recordings) on the course webpage as learning resources for future students. Additionally, we may use anonymized project information for internal statistical analysis of course outcomes. Please indicate your preferences below. *Your choices will not affect your grade in any way.*

Consent for Sharing Project Materials

Please keep the item you wish and remove the other one.

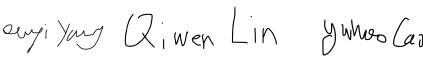
- All group members consent to allow the project materials (report, slides, and presentation recording) to be shared on the course page for future students.

Consent for Use of Project Information in Statistical Analysis

Please keep the item you wish and remove the other one.

- All group members consent to allow anonymized information about the project (e.g., topic, methods, outcomes, grades) to be used by the instructor for statistical analysis and course improvement.

Group Identification

- Group number: 11
- Names of group members: Ouyi Yang, Qiwen Lin, Yuhao Cao
- Signature of group members: 
- Date: 2026/4/10